

Ejercicio 14. Ordenamiento de panqueques arbitrarios mediante QuickSort e inversiones gpt_6_astra

Idea. Elegimos un pivote uniformemente al azar y usamos dos particiones binarias para formar los bloques

$$[P_i < v] [P_i = v] [P_i > v].$$

Ordenamos recursivamente el primero y el tercero. El bloque central contiene al pivote, por lo que ambas llamadas reciben menos elementos.

Tomamos $\log = \log_2$. Usaremos la caja negra $\text{Binario}(P, l, r, \lambda)$: ordena el segmento según etiquetas $\lambda(P_i) \in \{1, 2\}$ mediante inversiones, con tiempo y energía $O(m \log m)$ para $m \geq 2$, donde $m = r - l + 1$. Si $m \leq 1$, no realiza inversiones.

Las etiquetas son *virtuales*: cada consulta devuelve $\lambda(P_i)$ y cada inversión se ejecuta sobre los panqueques reales. Una inversión local de posiciones $u, \dots, w$ corresponde a

$$\text{flip}(l + u - 1, l + w - 1).$$

Su energía es $(l + w - 1) - (l + u - 1) = w - u$; por tanto, esta implementación conserva el costo de la caja negra.

Pseudocódigo. Entrada: $P[1..n]$, con tamaños positivos. Salida: el mismo arreglo ordenado de menor a mayor. Los índices comienzan en 1.

1. Ejecutar $\text{Ordenar}(P, 1, n)$.

$\text{Ordenar}(P, l, r)$:

1. Si $l \geq r$, regresar.
2. Elegir q uniformemente en $\{l, \dots, r\}$ y guardar $v \leftarrow P_q$.
3. Recorrer $P[l..r]$ y calcular

$$a \leftarrow \#\{i : P_i < v\}, \quad e \leftarrow \#\{i : P_i = v\}.$$

4. Ejecutar $\text{Binario}(P, l, r, \lambda)$, donde

$$\lambda(x) = \begin{cases} 1, & x < v, \\ 2, & x \geq v. \end{cases}$$

5. Ejecutar $\text{Binario}(P, l + a, r, \lambda)$, ahora con

$$\lambda(x) = \begin{cases} 1, & x = v, \\ 2, & x > v. \end{cases}$$

Esta llamada solo recibe elementos mayores o iguales que v .

6. Ejecutar $\text{Ordenar}(P, l, l + a - 1)$.
7. Ejecutar $\text{Ordenar}(P, l + a + e, r)$.

Correctitud. Demostramos por inducción sobre m que cada llamada ordena su segmento y conserva sus elementos.

1. **Base:** si $m \leq 1$, el segmento ya está ordenado y no se modifica.

2. **Hipótesis:** toda llamada sobre menos de m elementos ordena correctamente y conserva sus elementos.
3. **Paso:** sea $m \geq 2$. Por la garantía de la caja negra, el paso 4 coloca los a elementos menores que v en las primeras a posiciones. El sufijo contiene únicamente elementos mayores o iguales que v .

El paso 5 ordena las etiquetas de ese sufijo: sus primeros e elementos son iguales que v y los restantes son mayores. Por tanto, quedan exactamente los bloques

$$\underbrace{P[l..l+a-1]}_{<v}, \quad \underbrace{P[l+a..l+a+e-1]}_{=v}, \quad \underbrace{P[l+a+e..r]}_{>v}.$$

Como el pivote pertenece al segmento, $e \geq 1$; así,

$$0 \leq a \leq m-1, \quad 0 \leq m-a-e \leq m-1.$$

Por la hipótesis inductiva, las dos llamadas recursivas ordenan los bloques exteriores. El bloque central ya está ordenado porque todos sus elementos valen v . Las desigualdades entre bloques garantizan que su concatenación está ordenada.

Finalmente, $\text{flip}(i, j)$ mueve cada posición $i+t$ a $j-t$, para $0 \leq t \leq j-i$, y deja fijas las posiciones exteriores. Esta correspondencia es biyectiva; por ello, cada inversión conserva todos los panqueques.

La disminución estricta de los tamaños garantiza terminación, incluso si todos los tamaños son iguales.

Costo de una llamada. Para $m \geq 2$, elegir el pivote cuesta $O(1)$ y contar los elementos cuesta $O(m)$. La primera partición cuesta $O(m \log m)$; la segunda procesa a lo sumo m elementos y tiene la misma cota. Como $\log m \geq 1$, existen constantes $c_T, c_E > 0$ tales que los costos locales satisfacen

$$C_T(P, v) \leq c_T m \log m, \quad C_E(P, v) \leq c_E m \log m.$$

El conteo no consume energía porque no realiza inversiones.

Si $P_<$ y $P_>$ denotan los bloques recursivos, las recurrencias, condicionadas al pivote elegido, son

$$\begin{aligned} T(P) &= T(P_<) + T(P_>) + C_T(P, v), \\ E(P) &= E(P_<) + E(P_>) + C_E(P, v). \end{aligned}$$

Sus tamaños son a y $m-a-e$; para $m \leq 1$, el tiempo es constante y la energía es cero.

Análisis esperado del árbol recursivo. Justificaremos que la suma esperada de los tamaños de sus nodos internos pertenece a $O(n \log n)$.

Para analizar una llamada de tamaño $m \geq 2$, ordenamos conceptualmente sus panqueques por tamaño y rompemos empates mediante su identidad original. Esto asigna rangos distintos $1, \dots, m$ sin modificar el algoritmo. Como el pivote se elige uniformemente entre los panqueques, su rango k es uniforme.

Llamamos *bueno* a una elección cuyo rango satisface

$$\left\lceil \frac{m}{4} \right\rceil \leq k \leq \left\lfloor \frac{3m}{4} \right\rfloor + 1.$$

Este intervalo está contenido en $\{1, \dots, m\}$ para $m \geq 2$. Su número de rangos es

$$\left\lfloor \frac{3m}{4} \right\rfloor - \left\lceil \frac{m}{4} \right\rceil + 2.$$

Usando $\lfloor x \rfloor \geq x-1$ y $\lceil x \rceil \leq x+1$, dicho número es al menos $m/2$; por tanto, la probabilidad condicional de una elección buena es al menos $1/2$.

Hay a lo sumo $k - 1$ elementos estrictamente menores y a lo sumo $m - k$ estrictamente mayores que el pivote. Para una elección buena,

$$a \leq k - 1 \leq \frac{3m}{4}, \quad m - a - e \leq m - k \leq \frac{3m}{4}.$$

Así, cualquier hijo tiene tamaño a lo sumo $3m/4$.

Fijemos un panqueque i y sea D_i el número de llamadas de tamaño al menos 2 que lo contienen. Para $n \geq 2$, definimos

$$K = \left\lceil \log_{4/3} n \right\rceil.$$

Este valor basta porque, después de K elecciones buenas en su trayectoria, el tamaño satisface

$$m \leq n(3/4)^K \leq 1.$$

Las demás elecciones tampoco aumentan el tamaño.

Dividimos esa trayectoria en etapas: cada etapa termina con una elección buena o cuando el panqueque deja de participar. Hay a lo sumo K etapas. Si W es la duración de una etapa, tener $W > t$ exige que sus primeras t elecciones no sean buenas y que la trayectoria continúe. Como cada probabilidad condicional de fracaso es a lo sumo $1/2$,

$$\Pr(W > t) \leq 2^{-t}.$$

Para una variable entera no negativa, su esperanza es la suma de sus probabilidades de cola; por ello,

$$\mathbb{E}[W] = \sum_{t=0}^{\infty} \Pr(W > t), \quad \mathbb{E}[W] \leq \sum_{t=0}^{\infty} 2^{-t}, \quad \sum_{t=0}^{\infty} 2^{-t} = 2.$$

Sumando las etapas, sin requerir independencia, obtenemos $\mathbb{E}[D_i] \leq 2K$.

Sea m_u el tamaño del nodo interno u . Contar las parejas “panqueque contenido en un nodo” de ambas maneras da

$$\sum_u m_u = \sum_{i=1}^n D_i.$$

Por linealidad de la esperanza,

$$\mathbb{E} \left[\sum_u m_u \right] = \sum_{i=1}^n \mathbb{E}[D_i], \quad \mathbb{E} \left[\sum_u m_u \right] \leq 2nK.$$

Tiempo y energía totales. Cada nodo interno elimina de futuras llamadas al menos un panqueque de su bloque central. Estos bloques son disjuntos, de modo que hay a lo sumo n nodos internos y $2n + 1$ llamadas en total. El costo de los casos base pertenece, por tanto, a $O(n)$.

Como $m_u \leq n$, tenemos $\log m_u \leq \log n$. Sumando los costos locales,

$$T(P) \leq c_0 n + c_T \log n \sum_u m_u, \quad E(P) \leq c_E \log n \sum_u m_u.$$

Al tomar esperanzas,

$$\mathbb{E}[T(P)] \leq c_0 n + 2c_T n K \log n, \quad \mathbb{E}[E(P)] \leq 2c_E n K \log n.$$

Por cambio de base,

$$\log_{4/3} n = \frac{\log_2 n}{\log_2(4/3)},$$

de modo que $K \in O(\log n)$. Para $n \leq 1$, el tiempo es constante y la energía es cero.

Conclusión. El algoritmo ordena correctamente los panqueques con el mayor abajo, admite tamaños repetidos y satisface

$$\boxed{\mathbb{E}[T(P)] \in O(n \log^2 n) \quad \text{y} \quad \mathbb{E}[E(P)] \in O(n \log^2 n) .}$$

■